

```
// ER Score scoring engine – auto-extracted from the canonical single-flow protot
// (er-score-prototype.jsx) to keep the Vercel API in sync with the working refer
// Do not hand-edit the scoring logic here – update the prototype and re-extract
// or this file will silently drift out of sync the way it did previously.
```

```
/* =====
  SHARED SCORING ENGINE – full spec (ER-028 through ER-056)
  ===== */
```

```
function scoreCashFlow(takeHome, livingExpenses, newPayment) {
  const margin = takeHome - livingExpenses - newPayment;
  const pct = takeHome > 0 ? margin / takeHome : -1;
  let pts;
  if (pct > 0.30) pts = 40;
  else if (pct > 0.20) pts = 35;
  else if (pct > 0.15) pts = 30;
  else if (pct > 0.10) pts = 24;
  else if (pct > 0.05) pts = 18;
  else if (pct > 0.02) pts = 10;
  else if (pct >= 0) pts = 5;
  else pts = 0;
  return { pts, margin, pct };
}
```

```
function scoreSavings(remainingSavings, livingExpenses) {
  // ER-057: rescaled from 26-point to 24-point max
  const months = livingExpenses > 0 ? remainingSavings / livingExpenses : 99;
  let pts;
  if (months >= 12) pts = 24;
  else if (months >= 9) pts = 23;
  else if (months >= 6) pts = 22;
  else if (months >= 4) pts = 18;
  else if (months >= 3) pts = 14;
  else if (months >= 2) pts = 10;
  else if (months >= 1) pts = 5;
  else pts = 0;
  return { pts, months };
}
```

```
function scoreTaxableInvestments(investments, purchasePrice) {
  if (purchasePrice <= 0) return { pts: 5, ratio: null };
  const ratio = investments / purchasePrice;
  let pts;
```

```

    if (ratio >= 4) pts = 5;
    else if (ratio >= 3) pts = 4;
    else if (ratio >= 2) pts = 3;
    else if (ratio >= 1) pts = 2;
    else pts = 0;
    return { pts, ratio };
}

// ER-055 (graduated tiers) + ER-056 (years-until-target curve, replacing age-bas
function scoreRetirement(age, retirementBalance, income, targetRetirementAge, has
    // ER-079: Pension Adjustment - Effective Retirement Balance = account balance
    const pensionEquivalentValue = hasPension ? (annualPensionBenefit || 0) / 0.04
    const effectiveRetirementBalance = retirementBalance + pensionEquivalentValue;
    const target67 = (typeof targetRetirementAge === "number" && targetRetirementAg
    const yearsUntilTarget = Math.max(0, target67 - age);
    // curve keyed by years-until-target, mirroring Fidelity's age milestones (37/2
    const milestones = [[37, 1], [27, 3], [17, 6], [7, 8], [0, 10]];
    let targetMult;
    if (yearsUntilTarget >= milestones[0][0]) targetMult = milestones[0][1];
    else if (yearsUntilTarget <= milestones[milestones.length - 1][0]) targetMult =
    else {
        for (let i = 0; i < milestones.length - 1; i++) {
            const [y0, m0] = milestones[i];
            const [y1, m1] = milestones[i + 1];
            if (yearsUntilTarget <= y0 && yearsUntilTarget >= y1) {
                const frac = (y0 - yearsUntilTarget) / (y0 - y1);
                targetMult = m0 + frac * (m1 - m0);
                break;
            }
        }
    }
    const benchmarkTarget = targetMult * income;
    const pctOfBenchmark = benchmarkTarget > 0 ? effectiveRetirementBalance / bench
    // ER-055 graduated tiers
    let pts;
    // ER-057: rescaled from 4-point to 6-point max
    if (pctOfBenchmark >= 1.0) pts = 6;
    else if (pctOfBenchmark >= 0.75) pts = 5;
    else if (pctOfBenchmark >= 0.50) pts = 3;
    else if (pctOfBenchmark >= 0.25) pts = 2;
    else pts = 0;
    return { pts, pctOfBenchmark, target: benchmarkTarget, yearsUntilTarget, target
}

function scoreLoanSize(loanAmount, grossIncome, category, cash) {
    if (cash) return 7;
    const ratio = grossIncome > 0 ? loanAmount / grossIncome : 99;

```

```

    if (category === "consumer") {
        if (ratio <= 0.25) return 6; if (ratio <= 0.50) return 5; if (ratio <= 1.00)
    }
    if (category === "home") {
        if (ratio <= 2.00) return 6; if (ratio <= 3.00) return 5; if (ratio <= 4.00)
    }
    if (category === "investment") {
        if (ratio <= 3.00) return 6; if (ratio <= 4.50) return 5; if (ratio <= 6.00)
    }
    return 0;
}
function scoreDownPayment(downPct, cash) {
    if (cash) return 8;
    if (downPct >= 0.30) return 7; if (downPct >= 0.20) return 6; if (downPct >= 0.
}
function scoreLoanTerm(months, category, cash) {
    if (cash) return 5;
    if (category === "vehicle") { if (months <= 36) return 4; if (months <= 60) ret
    if (category === "recreational") { if (months <= 60) return 4; if (months <= 12
    if (category === "short_term") { if (months <= 24) return 4; if (months <= 48)
    if (category === "home") { const years = months / 12; if (years <= 15) return 4
    return 0;
}
function scoreLoanType(loanType, cash) {
    if (cash) return 5;
    return { fixed: 4, adjustable: 2, balloon: 1, interest_only: 0 }[loanType] ?? 2
}
// ER-077: HELOC Loan Size, based on % of Home Equity rather than income
function scoreHelocLoanSize(pctOfEquity) {
    if (pctOfEquity <= 0.50) return 7;
    if (pctOfEquity <= 0.70) return 5;
    if (pctOfEquity <= 0.85) return 3;
    return 0;
}

// ER-048: Cap Cost Reduction (Down Payment analog for leases) – inverted logic
function scoreCapCostReduction(pct) {
    if (pct <= 0) return 8; // required inception costs only
    if (pct <= 0.05) return 6; if (pct <= 0.10) return 4; if (pct <= 0.20) return 2
}
// ER-049: Monthly Lease Payment as % of gross monthly income (Loan Size analog f
function scoreLeasePayment(pctOfIncome) {
    if (pctOfIncome <= 0.10) return 7; if (pctOfIncome <= 0.15) return 5; if (pctOf
    if (pctOfIncome <= 0.25) return 1; return 0;
}
// ER-050: Lease Term
function scoreLeaseTerm(months) {

```

```

    // ER-073: top tier corrected from 4 to 5 to match the stated 5-point architect
    if (months <= 36) return 5; if (months <= 39) return 2; return 0;
}

// ER-047: Rental Readiness (replaces Financing Quality entirely for Renting)
function scoreRentalReadiness(remainingSavingsAfterUpfront, monthlyExpenses) {
    if (remainingSavingsAfterUpfront < 0) return { pts: 0, hardOverride: true, mont
    const months = monthlyExpenses > 0 ? remainingSavingsAfterUpfront / monthlyExpe
    let pts;
    if (months >= 3) pts = 25;
    else if (months >= 1) pts = 18;
    else pts = 10;
    return { pts, hardOverride: false, monthsCovered: months };
}

function estimateMonthlyPayment(principal, months, aprGuess = 0.075) {
    if (months <= 0) return 0;
    const r = aprGuess / 12;
    if (r === 0) return principal / months;
    return (principal * r) / (1 - Math.pow(1 + r, -months));
}

function monthsToPayoff(principal, annualRate, payment) {
    const r = annualRate / 12;
    if (principal <= 0) return 0;
    if (r === 0) return principal / payment;
    if (payment <= principal * r) return Infinity;
    return -Math.log(1 - (principal * r) / payment) / Math.log(1 + r);
}

function totalInterestPaid(payment, months, principal) {
    if (!isFinite(months)) return Infinity;
    return payment * months - principal;
}

function debtPayoffComparison(balance, annualRatePct, assumedMonths, cashApplied)
    const rate = annualRatePct / 100;
    const payment = estimateMonthlyPayment(balance, assumedMonths, rate);
    const originalInterest = totalInterestPaid(payment, assumedMonths, balance);
    const newBalance = Math.max(0, balance - cashApplied);
    const newMonths = monthsToPayoff(newBalance, rate, payment);
    const newInterest = totalInterestPaid(payment, newMonths, newBalance);
    return {
        payment, originalInterest, newBalance, newMonths,
        interestSaved: isFinite(newInterest) ? originalInterest - newInterest : null,
        monthsSaved: isFinite(newMonths) ? assumedMonths - newMonths : null,
    };
}

const PURCHASE_CATEGORIES = [

```

```

{ id: "vehicle", label: "Vehicle", financingCat: "consumer", termCat: "vehicle"
{ id: "boat", label: "Boat", financingCat: "consumer", termCat: "recreational",
{ id: "rv", label: "RV", financingCat: "consumer", termCat: "recreational", tra
{ id: "motorcycle", label: "Motorcycle", financingCat: "consumer", termCat: "re
{ id: "trailer", label: "Trailer", financingCat: "consumer", termCat: "recreati
{ id: "electronics", label: "Electronics", financingCat: "consumer", termCat: "
{ id: "furniture", label: "Furniture", financingCat: "consumer", termCat: "shor
{ id: "home_improvement", label: "Home Improvement", financingCat: "consumer",
{ id: "vacation", label: "Vacation", financingCat: "consumer", termCat: "short_
{ id: "wedding", label: "Wedding", financingCat: "consumer", termCat: "short_te
{ id: "education", label: "Education", financingCat: "consumer", termCat: "shor
{ id: "primary_residence", label: "Primary Residence", financingCat: "home", te
{ id: "investment_property", label: "Investment Property", financingCat: "inves
{ id: "renting", label: "Renting a Home", financingCat: "rental", termCat: null
{ id: "other", label: "Other", financingCat: "consumer", termCat: "short_term",
];

```

```

const BANDS = [
  { min: 80, max: 100, label: "Strong Financial Position", color: "#6FA37A", bg:
  { min: 70, max: 79, label: "Affordable with Meaningful Tradeoffs", color: "#D4A
  { min: 40, max: 69, label: "Significant Financial Impact", color: "#C97C3E", bg
  { min: 0, max: 39, label: "Limited Financial Position", color: "#C65B54", bg: "
];

```

```

function getBand(score) { return BANDS.find(b => score >= b.min && score <= b.max

```

```

/* =====

```

```

  MAIN COMPUTE FUNCTION

```

```

  p fields (superset across all purchase types):

```

```

  age, targetRetirementAge, annualIncome, takeHome, livingExpenses, savings,
  taxableInvestments, retirementBalance, delinquent,

```

```

  catDef, purchasePrice, payMethod ('cash' | 'finance' | 'lease' | 'credit_full' | 'cred
  cashDown, tradeInEquity, hasTradeIn, termMonths, interestRate, loanType, minPa
  estRent, rentalExpenses, ipMortgage,

```

```

  // Housing Situation (ER-051) – used when catDef.isHome or catDef.isRenting

```

```

  housingSituation: 'renting' | 'own_selling' | 'own_keeping' | 'none',

```

```

  currentMonthlyRent, currentHousingPayment,

```

```

  saleSalePrice, saleMortgageBalance, realtorPct, sellingClosingPct,

```

```

  newPITI, // used when new purchase is Primary Residence

```

```

  // Renting a Home (ER-047)

```

```

  monthlyRent, securityDeposit, lastMonthRequired,

```

```

  // Education (ER-052)

```

```

  eduLoanAmount, eduInterestRate, eduTermMonths, hasDeferment, defermentMonths,

```

```

  // Car Leasing (ER-048/049/050)

```

```

  negotiatedVehiclePrice, capCostReduction, monthlyLeasePayment, leaseTermMonths

```

```

===== */

```

```

function scoreDebtLoad(qualifyingDebt, annualIncome) {

```

```

// ER-067: graduated deduction applied to the TOTAL score, not a category compo
const ratio = annualIncome > 0 ? qualifyingDebt / annualIncome : 0;
if (ratio < 0.50) return 0;
if (ratio < 1.00) return -3;
if (ratio < 1.50) return -6;
if (ratio < 2.00) return -8;
return -10;
}

function computeScore(p) {
  const catDef = p.catDef;
  const isCash = p.payMethod === "cash" || p.payMethod === "credit_full";
  const isRevolving = p.payMethod === "credit_carry";
  const isLease = p.payMethod === "lease";
  const isHeloc = p.payMethod === "heloc";
  const isCashOutRefi = p.payMethod === "cashout_refi";

  const netSaleProceeds = (p.housingSituation === "own_selling")
    ? Math.max(0, p.saleSalePrice - p.saleMortgageBalance - p.saleSalePrice * (p.
      : 0;

  let effLivingExpenses = p.livingExpenses;
  let newPayment = 0;
  let totalDown = p.cashDown + (p.hasTradeIn ? p.tradeInEquity : 0);
  let loanAmount = Math.max(0, p.purchasePrice - totalDown);
  let downPct = p.purchasePrice > 0 ? totalDown / p.purchasePrice : 0;
  let cashSpent = 0; // amount that comes out of savings, for Financial Resilienc
  let rentalOverride = false;
  let savingsBoostFromSaleProceeds = 0;

  // --- Housing Situation (ER-051) applies to Primary Residence and Renting ---
  const housingApplies = catDef.isHome || catDef.isRenting;
  if (housingApplies) {
    if (p.housingSituation === "renting") {
      effLivingExpenses = Math.max(0, p.livingExpenses - (p.currentMonthlyRent ||
    } else if (p.housingSituation === "own_selling") {
      effLivingExpenses = Math.max(0, p.livingExpenses - (p.currentHousingPayment
      if (catDef.isHome) {
        totalDown = totalDown + netSaleProceeds; // proceeds reduce new loan / co
      } else if (catDef.isRenting) {
        savingsBoostFromSaleProceeds = netSaleProceeds; // proceeds flow into Sav
      }
    } else if (p.housingSituation === "own_keeping") {
      // no subtraction - both payments counted
    } else {
      // 'none' - no subtraction, default behavior
    }
  }
}

```

```

// --- Category-specific New Payment / Financing Quality-analog logic ---
let financingTotal, breakdown;

if (catDef.isInvestment && !isCashOutRefi) {
  const netRentalCF = p.estRent - p.rentalExpenses;
  const effectivePropCost = Math.max(0, p.ipMortgage - netRentalCF);
  newPayment = effectivePropCost;
  loanAmount = Math.max(0, p.purchasePrice - p.cashDown);
  downPct = p.purchasePrice > 0 ? p.cashDown / p.purchasePrice : 0;
  cashSpent = p.cashDown;
  const ls = scoreLoanSize(loanAmount, p.annualIncome, "investment", false);
  const dp = scoreDownPayment(downPct, false);
  const lt = scoreLoanTerm(p.termMonths, "home", false);
  const ty = scoreLoanType(p.loanType, false);
  financingTotal = ls + dp + lt + ty;
  breakdown = { ls, dp, lt, ty };
} else if (catDef.isRenting) {
  // ER-047: Rental Readiness replaces Financing Quality entirely
  const upfrontCost = p.monthlyRent + (p.lastMonthRequired ? p.monthlyRent : 0)
  newPayment = p.monthlyRent;
  cashSpent = upfrontCost; // raw upfront cost; savingsBoostFromSaleProceeds is
  const remainingAfterUpfront = p.savings + savingsBoostFromSaleProceeds - upfr
  const rr = scoreRentalReadiness(remainingAfterUpfront, effLivingExpenses);
  financingTotal = rr.pts;
  rentalOverride = rr.hardOverride;
  breakdown = { rentalReadiness: rr.pts, monthsCovered: rr.monthsCovered, upfro
  // Rental Readiness's own cash effect already captured via cashSpent below (u
} else if (housingApplies && catDef.isHome) {
  // Primary Residence
  newPayment = p.newPITI;
  cashSpent = p.cashDown; // trade-in/sale-proceeds equivalent already folded i
  loanAmount = Math.max(0, p.purchasePrice - totalDown);
  downPct = p.purchasePrice > 0 ? totalDown / p.purchasePrice : 0;
  const ls = scoreLoanSize(loanAmount, p.annualIncome, "home", false);
  const dp = scoreDownPayment(downPct, false);
  const lt = scoreLoanTerm(p.termMonths, "home", false);
  const ty = scoreLoanType(p.loanType, false);
  financingTotal = isCash ? 25 : (ls + dp + lt + ty);
  breakdown = isCash ? { ls: 7, dp: 8, lt: 5, ty: 5 } : { ls, dp, lt, ty };
  if (isCash) { newPayment = 0; cashSpent = p.purchasePrice; loanAmount = 0; }
} else if (isLease) {
  // ER-048/049/050: Car Leasing
  const price = p.negotiatedVehiclePrice || p.purchasePrice;
  let effMonthlyPayment = p.monthlyLeasePayment;
  if (p.isSinglePayLease) {
    effMonthlyPayment = p.leaseTermMonths > 0 ? p.totalLeaseCost / p.leaseTermM

```

```

    }
    newPayment = effMonthlyPayment;
    cashSpent = p.capCostReduction; // discretionary cap cost reduction paid from
    const capPct = price > 0 ? p.capCostReduction / price : 0;
    const grossMonthlyIncome = p.annualIncome / 12;
    const pctOfIncome = grossMonthlyIncome > 0 ? effMonthlyPayment / grossMonthly
    const ccr = scoreCapCostReduction(capPct);
    const mlp = scoreLeasePayment(pctOfIncome);
    const lt = scoreLeaseTerm(p.leaseTermMonths);
    const ty = 0; // N/A for leases (ER-050), scored 0 per Revolving Debt precede
    financingTotal = ccr + mlp + lt + ty;
    breakdown = { ls: mlp, dp: ccr, lt, ty };
} else if (isHeloc) {
    // ER-077: HELOC – equity-based Loan Size, N/A Down Payment, total term = dra
    const homeEquity = Math.max(0, p.homeValue - p.mortgageBalance);
    const pctOfEquity = homeEquity > 0 ? p.helocAmount / homeEquity : 1;
    const totalTermMonths = (p.drawPeriodYears + p.repaymentPeriodYears) * 12;
    const helocMonthlyPayment = p.loanType === "interest_only"
        ? p.helocAmount * (p.interestRate / 100 / 12)
        : estimateMonthlyPayment(p.helocAmount, totalTermMonths, p.interestRate / 1
    const remainingPayment = p.remainingAmountFinanced > 0 ? p.remainingMonthlyPa
    newPayment = helocMonthlyPayment + remainingPayment; // ER-077: remaining amo
    cashSpent = 0;
    loanAmount = p.helocAmount;
    downPct = 1; // N/A – scored as full points below, not derived from this rati
    const ls = scoreHelocLoanSize(pctOfEquity);
    const dp = 8; // N/A for HELOC – full points, no separate down payment concep
    const lt = scoreLoanTerm(totalTermMonths, "home", false);
    const ty = scoreLoanType(p.loanType, false);
    financingTotal = ls + dp + lt + ty;
    breakdown = { ls, dp, lt, ty };
} else if (isCashOutRefi) {
    // ER-084/087: Cash-Out Refinance – standard Home tables, all categories exce
    loanAmount = p.cashOutLoanAmount;
    downPct = 1; // N/A – full points, no separate down payment concept
    cashSpent = 0;
    const stdPayment = estimateMonthlyPayment(p.cashOutLoanAmount, p.termMonths,
    if (catDef.isInvestment) {
        // ER-087: nets against rental income like Investment Property's own mortga
        const netRentalCF = p.estRent - p.rentalExpenses;
        newPayment = Math.max(0, stdPayment - netRentalCF);
    } else {
        newPayment = stdPayment;
    }
}
const ls = scoreLoanSize(p.cashOutLoanAmount, p.annualIncome, "home", false);
const dp = 8; // N/A
const lt = scoreLoanTerm(p.termMonths, "home", false);

```



```

    const ty = scoreLoanType(p.loanType, false);
    financingTotal = ls + dp + lt + ty;
    breakdown = { ls, dp, lt, ty };
} else if (catDef.isEducation) {
    // ER-052: Education deferred payment
    const stdPayment = estimateMonthlyPayment(p.eduLoanAmount, p.eduTermMonths, p
    newPayment = (p.hasDeferment) ? 0 : stdPayment;
    cashSpent = 0; // education loans typically don't require cash down in this m
    loanAmount = p.eduLoanAmount;
    downPct = 0;

    const ls = scoreLoanSize(loanAmount, p.annualIncome, "consumer", false);
    const dp = scoreDownPayment(0, false);
    const lt = scoreLoanTerm(p.eduTermMonths, "short_term", false);
    const ty = scoreLoanType(p.loanType || "fixed", false);
    financingTotal = ls + dp + lt + ty;
    breakdown = { ls, dp, lt, ty, standardPayment: stdPayment };
} else if (isRevolving) {
    newPayment = p.minPayment;
    loanAmount = p.purchasePrice;
    downPct = 0;
    cashSpent = 0;

    const ls = scoreLoanSize(loanAmount, p.annualIncome, "consumer", false);
    const dp = scoreDownPayment(0, false);
    const lt = 0;
    const ty = 0; // ER-064: revolving debt scores 0 on Loan Type (was Adjustable
    financingTotal = ls + dp + lt + ty;
    breakdown = { ls, dp, lt, ty };
} else if (isCash) {
    newPayment = 0;
    loanAmount = 0;
    totalDown = p.purchasePrice;
    downPct = 1;
    cashSpent = p.purchasePrice;
    financingTotal = 25;
    breakdown = { ls: 7, dp: 8, lt: 5, ty: 5 };
} else {
    // standard installment loan (Vehicle/Recreational/Short-Term Consumer)
    newPayment = estimateMonthlyPayment(loanAmount, p.termMonths, p.interestRate
    cashSpent = p.cashDown;
    const ls = scoreLoanSize(loanAmount, p.annualIncome, catDef.financingCat, fal
    const dp = scoreDownPayment(downPct, false);
    const lt = scoreLoanTerm(p.termMonths, catDef.termCat, false);
    const ty = scoreLoanType(p.loanType, false);
    financingTotal = ls + dp + lt + ty;
    breakdown = { ls, dp, lt, ty };
}

```

```

// --- Cash Flow ---
const cf = scoreCashFlow(p.takeHome, effLivingExpenses, newPayment);

// --- Financial Resilience ---
const remainingSavings = Math.max(0, p.savings + savingsBoostFromSaleProceeds -
const savAdj = scoreSavings(remainingSavings, effLivingExpenses + newPayment);
const ti = scoreTaxableInvestments(p.taxableInvestments, p.purchasePrice);
const ret = scoreRetirement(p.age, p.retirementBalance, p.annualIncome, p.target
const cushionTotal = savAdj.pts + ti.pts + ret.pts;

// --- Master Scoring Formula (5.10, ER-072): categories -> Debt Load -> floor
const availableCashForTransaction = p.savings + netSaleProceeds; // ER-066
const baseScore = cf.pts + cushionTotal + financingTotal;

const qualifyingDebt = (p.studentLoans || 0) + (p.vehicleLoans || 0) + (p.credi
const debtLoadAdjustment = scoreDebtLoad(qualifyingDebt, p.annualIncome); // ER
const adjustedScore = Math.max(0, baseScore + debtLoadAdjustment); // ER-068 fl

let total = adjustedScore;
const overrides = [];
if (cf.margin < 0) overrides.push("Negative Cash Flow");
if (p.delinquent) overrides.push("Serious Delinquent Debt");
if (rentalOverride) overrides.push("Insufficient Rental Funds");
const impossiblePayment = (p.payMethod === "cash" || p.payMethod === "credit_fu
if (impossiblePayment) overrides.push("Impossible Payment Method");
const equityExceeded = isHeloc && p.helocAmount > Math.max(0, p.homeValue - p.m
if (equityExceeded) overrides.push("Equity-Exceeded");
const overridden = overrides.length > 0;
if (impossiblePayment || equityExceeded) total = 0;
else if (overridden) total = Math.min(total, 39);

return {
  cf, savAdj, ti, ret, cushionTotal, financingTotal, breakdown,
  total, overrides, overridden, loanAmount, downPct, totalDown, netSaleProceeds
  effLivingExpenses, newPayment, isCash, isRevolving, isLease, cashSpent, remai
  impossiblePayment, availableCashForTransaction, qualifyingDebt, debtLoadAdjus
};
}

function buildRecommendations(p, base) {
  const recs = [];
  const catDef = p.catDef;
  const skip = base.isCash || base.isRevolving || base.isLease || catDef.isRentin
  if (skip) return recs;

  const bump = Math.round(p.purchasePrice * 0.10);
  const alt1 = computeScore({ ...p, cashDown: p.cashDown + bump });
  if (alt1.total - base.total > 0) recs.push({ label: `Increase your cash down pa

```

```

const termOptions = { vehicle: [36,48,60,72,84,96], recreational: [60,84,120,180] };
const shortest = termOptions[0];
if (p.termMonths > shortest) {
  const alt2 = computeScore({ ...p, termMonths: shortest });
  if (alt2.total - base.total > 0) recs.push({ label: `Choose a shorter loan term` });
}
if (p.loanType !== "fixed") {
  const alt3 = computeScore({ ...p, loanType: "fixed" });
  if (alt3.total - base.total > 0) recs.push({ label: `Switch to a fixed-rate loan` });
}
const altPrice = Math.round(p.purchasePrice * 0.90);
const alt4 = computeScore({ ...p, purchasePrice: altPrice });
if (alt4.total - base.total > 0) recs.push({ label: `Reduce the purchase price` });

return recs.sort((a, b) => b.delta - a.delta).slice(0, 4);
}

module.exports = { computeScore, buildRecommendations, PURCHASE_CATEGORIES, getBaseScore };

```